

DATABRICKS MILESTONE 2

MCQs + Answers + Explanations

Extracted from the uploaded MCQ Master Preparation Guide — 120 questions

Only the practice MCQs, correct answers, and source explanations are included.

MODULE 1 — JAVA FUNDAMENTALS, CORE & COLLECTIONS — PRACTICE

1. Which component executes Java bytecode?

- A. JDK
- B. JRE only
- C. JVM
- D. javac

Answer: C — JVM

Explanation: The JVM executes bytecode; javac compiles source code.

2. Which command compiles App.java into bytecode?

- A. java App.java
- B. javac App.java
- C. jvm App.java
- D. run App.java

Answer: B — javac App.java

Explanation: javac is the Java compiler command.

3. For the assessment mental model, where are created objects generally allocated?

- A. Stack
- B. Heap
- C. CPU register only
- D. Source file

Answer: B — Heap

Explanation: Objects and arrays are generally treated as heap allocations.

4. What does autoboxing do?

- A. Wrapper to primitive
- B. Primitive to wrapper
- C. String to primitive
- D. Object to String

Answer: B — Primitive to wrapper

Explanation: Autoboxing converts a primitive value to its wrapper object.

5. Which collection is intended to model uniqueness?

- A. List
- B. Set
- C. Map
- D. Queue only

Answer: B — Set

Explanation: Set models unique elements.

6. Which statement about Map is correct?

- A. Map is a subtype of Collection
- B. Map stores key-value associations
- C. Map forbids duplicate values
- D. Map provides indexed access

Answer: B — Map stores key-value associations

Explanation: Map stores mappings from keys to values and is not a Collection subtype.

7. What happens when HashMap.put() uses an equal existing key?

- A. Adds a second key
- B. Rejects the value
- C. Replaces the mapping value
- D. Clears the map

Answer: C — Replaces the mapping value

Explanation: An equal key identifies the existing mapping, so its value is replaced.

8. Why must equals() and hashCode() be consistent for HashSet elements?

- A. For sorting
- B. For logical hashing and uniqueness
- C. For serialization
- D. For constructor chaining

Answer: B — For logical hashing and uniqueness

Explanation: Hash-based collections use hashing plus equality to locate logical duplicates.

9. What does List express?

- A. Only String values
- B. Number or an unknown subtype of Number
- C. Only Object values
- D. Only primitive numbers

Answer: B — Number or an unknown subtype of Number

Explanation: extends gives an upper bound: Number or an unknown subtype.

10. Which iterator is designed for bidirectional List traversal?

- A. Iterator
- B. ListIterator
- C. MapIterator
- D. StreamIterator

Answer: B — ListIterator

Explanation: ListIterator supports previous()/hasPrevious() as well as forward traversal.

11. What is the safe iterator operation for removing the last returned element?

- A. list.delete()
- B. collection.clear()
- C. iterator.remove()
- D. iterator.delete()

Answer: C — iterator.remove()

Explanation: Iterator.remove() coordinates the structural change with iterator state.

12. What is the result type of byte + byte in ordinary arithmetic?

- A. byte
- B. short
- C. int
- D. long

Answer: C — int

Explanation: Smaller integral operands are promoted to int for arithmetic.

13. What happens to the right side of false && expression?

- A. Always executes
- B. Does not execute
- C. Executes twice
- D. Throws by definition

Answer: B — Does not execute

Explanation: && short-circuits when the left operand is false.

14. What is $2 + 3 * 4$?

- A. 20
- B. 14
- C. 24
- D. 18

Answer: B — 14

Explanation: Multiplication has higher precedence than addition.

15. What is serialVersionUID used for?

- A. Database identity
- B. Serialization compatibility/version identification
- C. Thread ID
- D. Security token

Answer: B — Serialization compatibility/version identification

Explanation: It identifies the serialized class version for compatibility checks.

16. Which statement correctly contrasts static and instance members?

- A. Static belongs to each object
- B. Instance belongs to the class
- C. Static is class-level; instance belongs to an object
- D. They are identical

Answer: C — Static is class-level; instance belongs to an object

Explanation: static denotes class-level ownership; instance members belong to each object. MODULE 2 — JAVA OBJECT-ORIENTED PROGRAMMING Exam target: Learn the rule, recognize the trap, then prove it by solving the questions without looking at the answers. HIGH-YIELD RULES ■ A class defines structure/behavior; an object is an instance; constructors initialize newly created objects. ■ this() calls another constructor in the same class; super() calls a parent constructor. An explicit constructor call must be first. ■ Overloading = same name, different parameter list, compile-time choice. Overriding = subclass replacement of inherited instance behavior, runtime dispatch. ■ Parent reference + child object calls the child's overridden instance method. ■ An override cannot reduce visibility of the inherited method. ■ Inheritance models is-a. Composition models has-a and often reduces unnecessary coupling. ■ Interfaces define contracts/capabilities. Abstract classes may contain shared state and implementation and cannot be instantiated directly. ■ Encapsulation hides internal state and exposes controlled operations. ■ final variable: no reassignment; final method: no override; final class: no inheritance. ■ SRP means one primary reason to change. CODE PATTERNS TO RECOGNIZE

```
class Animal { void sound() { System.out.println("Animal"); } } class Dog extends Animal { @Override void sound() { System.out.println("Dog"); } } Animal a = new Dog(); a.sound(); // Dog class Product { String name; Product(String name) { this.name = name; } } class B extends A { B() { super(); System.out.println("B"); } }
```

 COMMON MCQ TRAPS ■ Changing only return type does not create a valid overload. ■ Static methods are hidden, not overridden in the same polymorphic sense as instance methods. ■ Do not narrow public/protected visibility when overriding. ■ Use composition for has-a rather than forcing inheritance. MUST-KNOW COMPARISONS Concept Remember Overloading / Overriding Different parameters, compile-time / inherited behavior replacement, runtime Inheritance / Composition is-a / has-a

MODULE 2 — JAVA OBJECT-ORIENTED PROGRAMMING — PRACTICE MCQs

1. A constructor differs from a method because a constructor:

- A. Must return int
- B. Has no return type
- C. Must be static
- D. Cannot take parameters

Answer: B — Has no return type

Explanation: Constructors initialize objects and do not declare a return type.

2. What must an explicit this() or super() constructor call be?

- A. Last statement
- B. First statement
- C. Inside a loop
- D. Optional after return

Answer: B — First statement

Explanation: Constructor chaining calls must be the first statement.

3. Which is overloading?

- A. Same signature in child
- B. Same name with different parameter list
- C. Changing only return type
- D. Changing only access

Answer: B — Same name with different parameter list

Explanation: Overloading uses different parameter lists and is resolved at compile time.

4. Changing only a method's return type creates a legal overload?

- A. Always
- B. Only for static methods
- C. No
- D. Only for final methods

Answer: C — No

Explanation: Return type alone is insufficient to distinguish overloaded signatures.

5. Parent reference points to child object and calls overridden instance method. Which implementation runs?

- A. Always parent
- B. Child implementation
- C. Compiler chooses randomly
- D. Both

Answer: B — Child implementation

Explanation: Runtime dispatch uses the actual object's overridden implementation.

6. What is runtime polymorphism primarily associated with?

- A. Overridden instance methods
- B. Only constructors
- C. Only static fields
- D. Primitive promotion

Answer: A — Overridden instance methods

Explanation: Overridden instance methods are dynamically dispatched.

7. Can an overriding method reduce visibility?

- A. Yes
- B. No
- C. Only if final
- D. Only if static

Answer: B — No

Explanation: An override cannot make inherited access less visible.

8. Which relationship is inheritance?

- A. Car has Engine
- B. Dog is Animal
- C. Order has List
- D. House has Room

Answer: B — Dog is Animal

Explanation: Inheritance models an is-a relationship.

9. Which relationship is composition?

- A. Dog is Animal
- B. Car has Engine
- C. Cat is Animal
- D. Circle is Shape

Answer: B — Car has Engine

Explanation: Composition models a has-a relationship.

10. What is a key purpose of an interface?

- A. Instantiate directly
- B. Define a contract/capability
- C. Replace all classes
- D. Store only fields

Answer: B — Define a contract/capability

Explanation: Interfaces define contracts implemented by classes.

11. Which statement about abstract classes is correct?

- A. They must be instantiated directly
- B. They cannot contain implementation
- C. They cannot be instantiated directly and may contain shared implementation
- D. They are identical to interfaces

Answer: C — They cannot be instantiated directly and may contain shared implementation

Explanation: Abstract classes can contain state and concrete behavior but cannot be instantiated directly.

12. Encapsulation commonly uses:

- A. Public mutable fields only
- B. Private state with controlled methods
- C. No methods
- D. Only static fields

Answer: B — Private state with controlled methods

Explanation: Encapsulation hides state and controls how it is changed.

13. A final method:

- A. Must be static
- B. Cannot be overridden
- C. Cannot be called
- D. Must be private

Answer: B — Cannot be overridden

Explanation: final prevents overriding.

14. What does this refer to?

- A. Immediate parent
- B. Current object
- C. Class loader
- D. Heap

Answer: B — Current object

Explanation: this refers to the current object.

15. What does super refer to conceptually?

- A. Current object only
- B. Immediate parent portion/behavior
- C. Child object
- D. Static JVM

Answer: B — Immediate parent portion/behavior

Explanation: super accesses parent constructor/implementation.

16. SRP says a class should:

- A. Have unlimited responsibilities
- B. Have one primary reason to change
- C. Never use methods
- D. Only contain static data

Answer: B — Have one primary reason to change

Explanation: Single Responsibility Principle focuses on one primary reason to change. MODULE 3 — JAVA FILE I/O & CORE APIs Exam target: Learn the rule, recognize the trap, then prove it by solving the questions without looking at the answers. HIGH-YIELD RULES ■ Byte streams such as FileInputStream/FileOutputStream are suited to raw binary data. ■ PrintWriter is convenient for formatted text output. ■ Try-with-resources automatically closes managed AutoCloseable resources. ■ Java Streams API is for declarative collection processing; it is not the same thing as file InputStream or Spark Streaming. ■ Pattern compiles regex; Matcher applies it. matches() checks the entire input; find() searches for a matching subsequence. ■ LocalDate = date only; LocalDateTime = date + time without zone; ZonedDateTime = date + time with zone context. ■ HttpClient sends HTTP requests; HttpRequest describes the request; HttpResponse carries the response. ■ transient excludes a field from default Java serialization. ■ Files.walk can traverse directories and returns a stream that should be closed appropriately. CODE PATTERNS TO RECOGNIZE try (BufferedReader br = Files.newBufferedReader(path)) { System.out.println(br.readLine()); } Pattern p = Pattern.compile("[A-Z]{3}\\d{3}"); Matcher m = p.matcher("ABC123"); boolean ok = m.matches(); // true LocalDate today = LocalDate.now(); LocalDate due = today.plusDays(7); HttpClient client = HttpClient.newHttpClient(); HttpRequest req = HttpRequest.newBuilder(URI.create(url)).GET().build(); HttpResponse res = client.send(req, HttpResponse.BodyHandlers.ofString()); COMMON MCQ TRAPS ■ Do not confuse Java Stream with InputStream or Spark Streaming. ■ matches() and find() are not interchangeable. ■ LocalDateTime does not carry a time-zone region. ■ transient is not access control and is not database persistence behavior. MUST-KNOW COMPARISONS Concept Remember Byte / Character I/O Binary bytes / text-oriented characters matches() / find() Whole input / matching subsequence LocalDate / LocalDateTime / ZonedDateTime Date / date+time / date+time+zone Stream / InputStream Collection processing abstraction / byte input stream

MODULE 3 — JAVA FILE I/O & CORE APIs — PRACTICE MCQs

1. Which is appropriate for raw binary image bytes?

- A. FileInputStream
- B. PrintWriter
- C. StringBuilder
- D. Pattern

Answer: A — FileInputStream

Explanation: Byte streams such as FileInputStream handle raw bytes.

2. What does FileOutputStream write?

- A. SQL rows
- B. Bytes
- C. Only JSON objects
- D. Regex patterns

Answer: B — Bytes

Explanation: FileOutputStream writes bytes.

3. What is PrintWriter convenient for?

- A. Formatted text output
- B. Binary encryption
- C. Heap allocation
- D. RDD creation

Answer: A — Formatted text output

Explanation: PrintWriter provides print/println/printf-style text output.

4. Main benefit of try-with-resources?

- A. Prevents all exceptions
- B. Automatically closes managed resources
- C. Makes code static
- D. Disables IO

Answer: B — Automatically closes managed resources

Explanation: AutoCloseable resources are closed when the block exits.

5. Which is an intermediate Java Stream operation?

- A. filter
- B. collect/toList
- C. forEach only
- D. close

Answer: A — filter

Explanation: filter transforms a stream pipeline and is intermediate.

6. What does Pattern do in regex API?

- A. Executes SQL
- B. Compiles a regex pattern
- C. Stores files
- D. Creates threads

Answer: B — Compiles a regex pattern

Explanation: Pattern represents a compiled regex.

7. Matcher.matches() generally means:

- A. Search any substring
- B. Match the entire input
- C. Replace all text
- D. Split input

Answer: B — Match the entire input

Explanation: matches attempts to match the entire region/input.

8. Matcher.find() generally means:

- A. Validate entire input only
- B. Search for matching subsequences
- C. Compile pattern
- D. Close matcher

Answer: B — Search for matching subsequences

Explanation: find searches for the next matching subsequence.

9. Which java.time type carries time-zone context?

- A. LocalDate
- B. LocalDateTime
- C. ZonedDateTime
- D. Month

Answer: C — ZonedDateTime

Explanation: ZonedDateTime includes zone/offset context.

10. What does HttpClient do?

- A. Stores files
- B. Sends HTTP requests and receives responses
- C. Creates JVM bytecode
- D. Parses regex only

Answer: B — Sends HTTP requests and receives responses

Explanation: HttpClient is the HTTP request/response API.

11. What does transient do in default Java serialization?

- A. Encrypts field
- B. Skips field from default serialized state
- C. Makes field static
- D. Makes field final

Answer: B — Skips field from default serialized state

Explanation: transient fields are excluded from default serialization.

12. Which API can traverse a directory tree as a stream of paths?

- A. Files.walk
- B. Math.walk
- C. Pattern.walk
- D. Thread.walk

Answer: A — Files.walk

Explanation: Files.walk returns a stream of paths. MODULE 4 — JAVA CONCURRENCY, JDBC & ADVANCED UTILITIES Exam target: Learn the rule, recognize the trap, then prove it by solving the questions without looking at the answers. HIGH-YIELD RULES ■ Race condition = timing/interleaving affects a shared mutable state result. ■ Deadlock = threads can wait forever for locks held by one another; consistent global lock ordering reduces circular-wait risk. ■ ConcurrentHashMap is designed for concurrent map access; compound operations may still need atomic map methods. ■ CopyOnWriteArrayList is best conceptually for read-heavy, write-rare workloads. ■ CompletableFuture represents asynchronous computations that can be chained or combined. ■ A JDBC transaction groups related changes so they succeed or fail as a unit. commit() makes changes permanent; rollback() undoes uncommitted work. ■ setAutoCommit(false) is commonly used when explicitly controlling a multi-step transaction. ■ Connection pooling reuses physical DB connections; close() on a pooled connection generally returns it to the pool. CODE PATTERNS TO RECOGNIZE conn.setAutoCommit(false); try { // update A // update B conn.commit(); } catch (SQLException e) { conn.rollback(); } CompletableFuture.supplyAsync() -> loadData() .thenApply(data -> summarize(data)); COMMON MCQ TRAPS ■ Race condition and deadlock are different failure modes. ■ CopyOnWriteArrayList is not a good default for write-heavy workloads. ■ rollback() does not commit; it undoes uncommitted transaction work. ■ Do not keep borrowed pooled connections forever. FAILURE → MECHANISM MAP Scenario Think of Timing-dependent wrong shared-state result Race condition Threads waiting in a cycle Deadlock; consistent lock ordering Many reads, rare writes CopyOnWriteArrayList Concurrent map access ConcurrentHashMap Async dependent computation CompletableFuture Related DB changes must succeed/fail together Transaction + commit/rollback Repeated physical DB connection creation Connection pooling

MODULE 4 — JAVA CONCURRENCY, JDBC & ADVANCED UTILITIES —

1. Two threads read counter=10, both write 11. What is this?

- A. Deadlock
- B. Race condition/lost update
- C. Serialization
- D. Pooling

Answer: B — Race condition/lost update

Explanation: Timing-dependent shared-state updates can lose increments.

2. Thread A locks X then waits for Y; Thread B locks Y then waits for X. Risk?

- A. Autoboxing
- B. Deadlock
- C. Overloading
- D. Checkpoint

Answer: B — Deadlock

Explanation: Circular waiting creates deadlock risk.

3. Which collection fits many reads and rare writes?

- A. CopyOnWriteArrayList
- B. HashMap always
- C. ArrayList only
- D. Stack

Answer: A — CopyOnWriteArrayList

Explanation: Copy-on-write favors read-heavy, write-rare workloads.

4. Which map is designed for concurrent access by multiple threads?

- A. HashMap only
- B. ConcurrentHashMap
- C. TreeMap only
- D. WeakHashMap only

Answer: B — ConcurrentHashMap

Explanation: ConcurrentHashMap is designed for concurrent map access.

5. What is CompletableFuture mainly for?

- A. Binary files
- B. Asynchronous computation/chaining
- C. Regex compilation
- D. RDD lineage

Answer: B — Asynchronous computation/chaining

Explanation: It represents asynchronous computations that can be composed.

6. Why use JDBC setAutoCommit(false) for related updates?

- A. To disable SQL
- B. To group changes into a transaction
- C. To create a pool
- D. To enable regex

Answer: B — To group changes into a transaction

Explanation: It lets related changes commit or roll back as a unit.

7. What does rollback() do?

- A. Makes changes permanent
- B. Undoes uncommitted work
- C. Creates connection
- D. Commits automatically

Answer: B — Undoes uncommitted work

Explanation: rollback reverses uncommitted transaction work.

8. What problem does connection pooling address?

- A. Regex matching
- B. Repeated physical DB connection creation
- C. Object inheritance
- D. RDD shuffles

Answer: B — Repeated physical DB connection creation

Explanation: Pools reuse database connections instead of repeatedly creating them. MODULE 5 — SCALA BASICS & DATA STRUCTURES Exam target: Learn the rule, recognize the trap, then prove it by solving the questions without looking at the answers. HIGH-YIELD RULES ■ val cannot be rebound; var can be reassigned. ■ Type inference lets Scala infer many variable/expression types. ■ Tuples group a fixed number of values and support positional access/destructuring. ■ Array is indexed and mutable; immutable List operations return new values. ■ :: prepends to a List and returns a new List. ■ Pattern matching checks ordered cases; _ is a wildcard fallback. ■ A guard adds an if-condition to a case. ■ map produces one output element per input element. ■ A lambda is an anonymous function value. CODE PATTERNS TO RECOGNIZE val tax = 0.18 var attempts = 0 attempts += 1 val order = (101, "Chennai", 2500.0) val (id, city, amount) = order val xs = List(2, 3) val ys = 1 :: xs // xs = List(2,3) // ys = List(1,2,3) val label = status match { case 200 => "OK" case 404 => "Missing" case _ => "Other" } COMMON MCQ TRAPS ■ val does not make a referenced mutable object deeply immutable. ■ Do not treat Array and immutable List as equivalent. ■ case order matters in pattern matching. ■ :: does not mutate the original List. MUST-KNOW COMPARISONS Concept Remember val / var Cannot rebound / can reassign Array / immutable List Indexed mutable / immutable collection :: Prepends and returns a new List match / guard Pattern selection / extra if-condition

MODULE 5 — SCALA BASICS & DATA STRUCTURES — PRACTICE MCQs

1. Which Scala binding cannot be reassigned?

- A. var
- B. val
- C. let
- D. const

Answer: B — val

Explanation: val cannot be rebound.

2. Which can be reassigned?

- A. val
- B. var
- C. case
- D. tuple

Answer: B — var

Explanation: var is assignable.

3. What does Scala type inference do?

- A. Deletes types
- B. Infers many types from expressions/values
- C. Converts Scala to JavaScript always
- D. Creates RDDs

Answer: B — Infers many types from expressions/values

Explanation: Scala can infer many expression and variable types.

4. How many values can a tuple contain conceptually?

- A. Exactly two
- B. A fixed number of values
- C. Only one
- D. Only strings

Answer: B — A fixed number of values

Explanation: Tuples group a fixed number of potentially different types.

5. Which structure here is naturally mutable?

- A. Immutable List
- B. Array
- C. Option
- D. Tuple

Answer: B — Array

Explanation: Scala Array elements can be updated.

6. What does :: do to a List?

- A. Mutates the existing List
- B. Prepends and returns a new List
- C. Sorts the List
- D. Deletes the head

Answer: B — Prepends and returns a new List

Explanation: :: prepends immutably.

7. Which is true of an immutable List?

- A. Elements are updated in place
- B. Operations return new values
- C. It cannot be traversed
- D. It is always an Array

Answer: B — Operations return new values

Explanation: Immutable operations derive new collections.

8. In pattern matching, case order matters because:

- A. Only the last case runs
- B. First matching case is selected
- C. All cases run
- D. Cases are random

Answer: B — First matching case is selected

Explanation: Patterns are checked in order.

9. What does a pattern guard add?

- A. A second class
- B. An if-condition to a case
- C. A database
- D. A new JVM

Answer: B — An if-condition to a case

Explanation: A guard requires both the pattern and condition to match.

10. What does `List(1,2,3).map(_ * 10)` produce?

- A. `List(1,2,3)`
- B. `List(10,20,30)`
- C. 10
- D. `List(2,3,4)`

Answer: B — `List(10,20,30)`

Explanation: `map` transforms each element.

11. What is a lambda?

- A. Anonymous function value
- B. A database table
- C. A JVM process
- D. A mutable List

Answer: A — Anonymous function value

Explanation: A lambda is an anonymous function value.

12. What does tuple destructuring do?

- A. Deletes tuple values
- B. Binds tuple positions to variables
- C. Converts tuple to RDD
- D. Sorts fields

Answer: B — Binds tuple positions to variables

Explanation: Pattern/destructuring binds tuple components to names. MODULE 6 — SCALA FUNCTIONAL TRANSFORMATIONS Exam target: Learn the rule, recognize the trap, then prove it by solving the questions without looking at the answers. HIGH-YIELD RULES ■ map keeps one output per input element. ■ flatMap maps and then flattens nested results; use it when one input can produce zero, one, or many outputs. ■ filter retains elements satisfying a predicate. ■ Transformation order matters; filtering early can reduce later work when logically valid. ■ Higher-order functions accept a function or return a function. ■ Pure functions depend on explicit inputs and avoid hidden side effects. ■ Immutability means deriving new values rather than mutating shared state. ■ Functional pipelines are sequences of composable transformations. CODE PATTERNS TO RECOGNIZE `List(1,2,3).map(_ * 10)` // `List(10,20,30)` `val lines = List("a b", "c")` `lines.flatMap(_.split(" "))` // `List("a","b","c")` `records.filter(_.active).flatMap(_.contacts).filter(_.verified)` COMMON MCQ TRAPS ■ `map(_.split(" "))` does not flatten the nested results. ■ flatMap is not simply a synonym for filter. ■ `A => Boolean` is a predicate type. ■ Pure does not mean 'never computes'; it means output is determined by explicit inputs without hidden side effects. PIPELINE SHAPE Operation Shape `map` 1 input → 1 output `filter` 1 input → kept or removed `flatMap` 1 input → 0/many outputs → flattened

MODULE 6 — SCALA FUNCTIONAL TRANSFORMATIONS — PRACTICE MCQs

1. When is flatMap preferred to map?

- A. When every input must produce exactly one output
- B. When one input may produce zero/many outputs and results should be flattened
- C. Only for sorting
- D. Only for numbers

Answer: B — When one input may produce zero/many outputs and results should be flattened

Explanation: flatMap maps and flattens nested results.

2. What does filter do?

- A. Transforms every element
- B. Retains elements satisfying a predicate
- C. Always sorts
- D. Flattens nested collections

Answer: B — Retains elements satisfying a predicate

Explanation: filter keeps elements for which the predicate is true.

3. A predicate from A to Boolean has type:

- A. $A \Rightarrow B$
- B. $A \Rightarrow \text{Boolean}$
- C. $\text{Boolean} \Rightarrow A$
- D. $A \Rightarrow \text{Unit}$

Answer: B — $A \Rightarrow \text{Boolean}$

Explanation: A predicate maps an A to Boolean.

4. A transformation $A \Rightarrow B$ means:

- A. A is input and B is output
- B. B is always Boolean
- C. A is output only
- D. No function exists

Answer: A — A is input and B is output

Explanation: The function consumes A and returns B.

5. Why can filtering early improve a valid pipeline?

- A. It increases input size
- B. It reduces rows processed by later work
- C. It disables Spark
- D. It forces a shuffle

Answer: B — It reduces rows processed by later work

Explanation: Fewer records reach later transformations.

6. A pure function's result depends on:

- A. Hidden global state
- B. Explicit inputs
- C. Random thread timing
- D. Database state only

Answer: B — Explicit inputs

Explanation: Pure functions are determined by explicit inputs and avoid hidden side effects.

7. What is immutability?

- A. Changing shared state in place
- B. Deriving new values rather than mutating existing shared state
- C. Deleting all data
- D. Using only var

Answer: B — Deriving new values rather than mutating existing shared state

Explanation: Immutable style derives new values.

8. What is a higher-order function?

- A. A function that accepts/returns functions
- B. A function with no parameters
- C. A JVM method only
- D. A database function

Answer: A — A function that accepts/returns functions

Explanation: Higher-order functions operate on functions as values. MODULE 7 — SPARK RDDs Exam target: Learn the rule, recognize the trap, then prove it by solving the questions without looking at the answers. HIGH-YIELD RULES ■ parallelize creates an RDD from a local collection; textFile creates an RDD from external text data. ■ Transformations define new RDDs lazily; actions trigger execution and return/write results. ■ Typical narrow transformations include map/filter; key-based aggregations and joins can require shuffle. ■ collect() returns all elements to the driver; it is dangerous for large datasets. ■ Lineage records transformation dependencies and supports recomputation of lost partitions. ■ Pair RDDs contain key-value tuples. reduceByKey aggregates values per key. ■ join combines Pair RDD records by key and can cause shuffle. ■ persist/cache keeps computed partitions for reuse; cache is a persistence choice, not a recovery checkpoint. ■ MEMORY_AND_DISK conceptually provides memory storage with disk fallback. ■ mapPartitions invokes a function once per partition and can reduce repeated setup overhead. CODE PATTERNS TO RECOGNIZE `val rdd = sc.parallelize(Seq(1,2,3,4), 2)` `val adults = people.filter(_._age >= 18)` `val n = adults.count()` // action triggers execution `val totals = sales .map(x => (x.region, x.amount)) .reduceByKey(_ + _)` `val cleaned = raw.filter(valid).persist()` `cleaned.count()` `cleaned.saveAsTextFile(out)` COMMON MCQ TRAPS ■ Creating a transformation does not mean the data has already been fully processed. ■ collect() is not a safe default for large RDDs. ■ Lineage is dependency history, not the same thing as checkpoint data. ■ Do not cache everything; persistence is justified by reuse. RDD DECISION MAP Need Likely answer Create from local collection parallelize Read external text textFile Define new RDD Transformation Trigger execution Action Per-key aggregation reduceByKey Combine Pair RDDs by key join Reuse expensive intermediate persist/cache Recovery/progress metadata checkpoint (where applicable) Small sample to driver take() All records to driver collect() — risky for large data

MODULE 7 — SPARK RDDs — PRACTICE MCQs

1. What does `sc.parallelize()` do?

- A. Reads JDBC
- B. Creates an RDD from a local collection
- C. Creates a DataFrame only
- D. Writes Parquet

Answer: B — Creates an RDD from a local collection

Explanation: `parallelize` distributes a local collection into an RDD.

2. Which is a common external-source RDD API for text?

- A. `sc.textFile`
- B. `sc.mapFile`
- C. `sc.sqlFile`
- D. `sc.jdbcRDDOnly`

Answer: A — `sc.textFile`

Explanation: `textFile` reads text data into an RDD.

3. Which is a transformation?

- A. `count`
- B. `collect`
- C. `filter`
- D. `saveAsTextFile`

Answer: C — `filter`

Explanation: `filter` defines a new RDD lazily.

4. Which is an action?

- A. `map`
- B. `filter`
- C. `count`
- D. `flatMap`

Answer: C — `count`

Explanation: `count` triggers execution and returns a result.

5. When are RDD transformations evaluated?

- A. Immediately
- B. Lazily when an action requires execution
- C. Only at compile time
- D. Never

Answer: B — Lazily when an action requires execution

Explanation: Spark evaluates transformations when an action triggers execution.

6. What does lineage represent?

- A. Database schema
- B. Dependency chain used to derive an RDD
- C. Only cached data
- D. JVM stack

Answer: B — Dependency chain used to derive an RDD

Explanation: Lineage records the logical transformation dependencies.

7. Why is lineage useful?

- A. It enables recomputation of lost partitions
- B. It prevents all shuffles
- C. It replaces schema
- D. It encrypts data

Answer: A — It enables recomputation of lost partitions

Explanation: Spark can recompute lost partitions from lineage.

8. Why can collect() be dangerous?

- A. It deletes the RDD
- B. It brings all records to the driver
- C. It always sorts
- D. It disables caching

Answer: B — It brings all records to the driver

Explanation: All elements are returned to driver memory.

9. What is a narrow transformation?

- A. Requires broad redistribution
- B. Each output partition depends on limited input partitions
- C. Always writes to disk
- D. Only applies to DataFrames

Answer: B — Each output partition depends on limited input partitions

Explanation: map/filter are typical narrow transformations.

10. Which operation commonly involves a shuffle?

- A. map
- B. filter
- C. reduceByKey
- D. mapValues only

Answer: C — reduceByKey

Explanation: Per-key aggregation commonly redistributes data by key.

11. What does reduceByKey do?

- A. Aggregates values for each key
- B. Sorts all keys globally
- C. Removes keys
- D. Creates a DataFrame

Answer: A — Aggregates values for each key

Explanation: It combines values for the same key.

12. What does Pair RDD join conceptually do?

- A. Joins records by key
- B. Joins by array index only
- C. Removes duplicate keys
- D. Caches all data

Answer: A — Joins records by key

Explanation: Pair RDD joins combine records sharing keys.

13. When should an RDD be persisted?

- A. Always
- B. When an expensive intermediate is reused
- C. Only before first transformation
- D. Never

Answer: B — When an expensive intermediate is reused

Explanation: Persistence avoids repeated recomputation for reused data.

14. What does MEMORY_AND_DISK conceptually provide?

- A. Memory only
- B. Memory storage with disk fallback
- C. Disk deletion
- D. No storage

Answer: B — Memory storage with disk fallback

Explanation: It can keep partitions in memory and spill/use disk when needed.

15. What does mapPartitions provide?

- A. Function once per partition with an iterator
- B. Only one function for the entire cluster
- C. Automatic SQL
- D. Only sorting

Answer: A — Function once per partition with an iterator

Explanation: It can share setup across records in a partition.

16. Which is a sensible use of take(5)?

- A. Retrieve a small sample to the driver
- B. Move all data to driver
- C. Persist forever
- D. Force a shuffle

Answer: A — Retrieve a small sample to the driver

Explanation: take limits the amount returned to the driver. MODULE 8 — SCALA FILE I/O & ERROR-HANDLING TYPES Exam target: Learn the rule, recognize the trap, then prove it by solving the questions without looking at the answers. HIGH-YIELD RULES ■ scala.io.Source is a basic text input utility. ■ Source resources should be closed after use. ■ mkString combines content into one String and is convenient for small complete reads. ■ Option = Some(value) or None; use it for expected absence. ■ Try = Success(value) or Failure(exception); use it for exception-producing computations. ■ recover can convert selected failures into fallback values. ■ Either commonly uses Right for success and Left for error information. CODE PATTERNS TO RECOGNIZE val src = scala.io.Source.fromFile(path) try { val text = src.mkString } finally { src.close() } val city: Option[String] = findCity(id) println(city.getOrElse("Unknown")) val n = scala.util.Try(text.toInt) val safe = n.getOrElse(0) Right(order) // success Left("invalid amount") // error COMMON MCQ TRAPS ■ Option is about presence/absence, not exception capture. ■ Try is about an exception-producing computation. ■ Either is useful when explicit error information is needed. ■ mkString is convenient for small content, not automatically the best approach for huge files. ERROR-HANDLING MAP Type Use when Option Expected presence/absence Try Computation may throw an exception Either Need explicit success/error information

MODULE 8 — SCALA FILE I/O & ERROR-HANDLING TYPES — PRACTICE

1. What is `scala.io.Source` used for?

- A. Reading text files/URLs
- B. Creating JVMs
- C. Writing binary databases
- D. RDD checkpointing

Answer: A — Reading text files/URLs

Explanation: Source is a basic Scala text input utility.

2. Why should Source be closed?

- A. To release the underlying resource
- B. To change the file name
- C. To compile Scala
- D. To sort lines

Answer: A — To release the underlying resource

Explanation: Closing releases the underlying file/resource handle.

3. What does `mkString` do?

- A. Combines elements/characters into one String
- B. Creates a database
- C. Creates a tuple
- D. Starts Spark

Answer: A — Combines elements/characters into one String

Explanation: `mkString` concatenates content into a String.

4. For a small complete text file, which is convenient?

- A. `mkString`
- B. `reduceByKey`
- C. `cache`
- D. `ZonedDateTime`

Answer: A — `mkString`

Explanation: `mkString` is convenient for complete small-content reads.

5. Option represents:

- A. Success/failure with exception
- B. Value present or absent
- C. Two explicit error channels only
- D. A Spark partition

Answer: B — Value present or absent

Explanation: Option models presence with `Some` or absence with `None`.

6. Try represents:

- A. A computation that may throw, as `Success/Failure`
- B. Only missing values
- C. A tuple
- D. A `DataFrame`

Answer: A — A computation that may throw, as `Success/Failure`

Explanation: Try captures exception-producing computations.

7. Either commonly uses Right for:

- A. Error only
- B. Success
- C. Missing file only
- D. Shuffle

Answer: B — Success

Explanation: Common convention: Right is success, Left is error.

8. Why prefer Either over Option for validation errors?

- A. Either can carry explicit error information
- B. Option always throws
- C. Either is mutable
- D. Option is a DataFrame

Answer: A — Either can carry explicit error information

Explanation: Either can preserve an error value/message in Left. MODULE 9 — SPARK DATAFRAMES WITH SCALA Exam target: Learn the rule, recognize the trap, then prove it by solving the questions without looking at the answers. HIGH-YIELD RULES ■ RDDs are low-level distributed collections; DataFrames are schema-aware, column-oriented structured APIs. ■ For structured tabular/JSON/Parquet and SQL-style analytics, DataFrames are generally preferred. ■ Explicit schemas declare expected names/types/nullability; inferSchema asks Spark to infer types. ■ An explicit schema improves predictability but does not make malformed source values valid. ■ Built-in DataFrame/Spark SQL expressions expose semantics Spark can analyze and optimize. ■ Common structured operations include col(), filter(), groupBy() and aggregation. ■ A typical Scala batch flow is SparkSession → read → transform → action/write. CODE PATTERNS TO RECOGNIZE `val schema = StructType(Seq( StructField("id", IntegerType, false), StructField("amount", DecimalType(10,2), true) ))` `val df = spark.read.schema(schema).csv(path)` `df .filter(col("amount") > 1000) .groupBy("region") .sum("amount")` `val df = spark.read.parquet(in)` `val out = df.filter($"active" === true)` `out.write.mode("overwrite").parquet(dest)` COMMON MCQ TRAPS ■ DataFrame does not mean no execution plan; Spark still builds one. ■ DecimalType does not magically convert arbitrary malformed text. ■ Do not choose a UDF automatically when a built-in expression can solve the problem. RDD vs DATAFRAME RDD DataFrame Low-level distributed collection Schema-aware structured API Custom/unstructured processing Structured tabular/SQL-style analytics Less structure for optimizer More structure/column semantics for optimization

MODULE 9 — SPARK DATAFRAMES WITH SCALA — PRACTICE MCQs

1. For structured tabular analytics, which is generally preferred?

- A. RDD by default
- B. DataFrame
- C. Thread
- D. Socket

Answer: B — DataFrame

Explanation: DataFrames provide schema-aware structured APIs and optimization opportunities.

2. What does an explicit schema provide?

- A. Predictable expected names/types
- B. Guaranteed clean source data
- C. Automatic deduplication
- D. No parsing

Answer: A — Predictable expected names/types

Explanation: It declares expected structure and types.

3. What does inferSchema do?

- A. Asks Spark to infer types from source data
- B. Forces all values to String
- C. Disables parsing
- D. Creates UDFs

Answer: A — Asks Spark to infer types from source data

Explanation: Schema inference examines data to guess types.

4. Does DecimalType guarantee malformed text becomes a valid decimal?

- A. Yes
- B. No
- C. Only in RDDs
- D. Only with cache

Answer: B — No

Explanation: Schema declaration does not make invalid source text valid.

5. Why prefer built-in DataFrame functions when possible?

- A. Spark can understand/analyze them better
- B. They always avoid all shuffles
- C. They disable schema
- D. They require no Spark

Answer: A — Spark can understand/analyze them better

Explanation: Built-ins expose structured semantics to Spark.

6. What does groupBy().sum() express?

- A. Structured aggregation
- B. RDD lineage only
- C. Thread synchronization
- D. Regex matching

Answer: A — Structured aggregation

Explanation: It expresses a DataFrame aggregation.

7. Which flow is sensible for a Scala batch Spark job?

- A. Write → read → SparkSession
- B. SparkSession → read → transform → write/action
- C. JDBC → constructor only
- D. RDD → JVM compile

Answer: B — SparkSession → read → transform → write/action

Explanation: This is the standard high-level batch flow.

8. What is a key advantage of DataFrame schema/columns?

- A. Named structure Spark can optimize
- B. No memory use
- C. No transformations
- D. No execution plan

Answer: A — Named structure Spark can optimize

Explanation: Schema and column semantics enable structured analysis/optimization. MODULE 10 — ADVANCED SPARK: PLANS, UDFs & EXTERNAL SOURCES Exam target: Learn the rule, recognize the trap, then prove it by solving the questions without looking at the answers. HIGH-YIELD RULES ■ Logical plan describes what the query means; physical plan describes how Spark will execute it. ■ `df.explain()` is the practical diagnostic tool for inspecting plans. ■ Exchange commonly indicates repartitioning/data redistribution and often corresponds to shuffle. ■ Shuffle is more expensive than purely local partition processing and is a common performance concern. ■ Prefer built-in Spark SQL functions over equivalent opaque UDFs when possible. ■ When logically valid, filter selective rows before an expensive UDF. ■ Partitioned JDBC reads use `partitionColumn`, `lowerBound`, `upperBound` and `numPartitions` conceptually. ■ `numPartitions` also limits concurrent JDBC partitions/connections. ■ Parquet supports column pruning; partition-column filters can enable partition pruning. ■ JSON is flexible for raw/semi-structured ingestion; Parquet is efficient for structured analytics. CODE PATTERNS TO RECOGNIZE `df.groupBy("region").sum("amount").explain(true)` // Prefer built-ins when possible `df.withColumn("name2", lower(col("name"))) // Filter first when logically valid df.filter(col("active") === true).withColumn("score", expensiveUdf(col("text"))) COMMON MCQ TRAPS ■ Exchange does not mean 'filter'; it points to redistribution/repartitioning. ■ UDF is not the default answer for every custom-looking transformation. ■ More JDBC partitions are not automatically better; the database must support the concurrency. ■ Column pruning and partition pruning are different optimizations. PERFORMANCE REASONING Clue Think Exchange in physical plan Redistribution/shuffle Equivalent built-in function exists Prefer built-in Only 10% of rows need expensive custom work Filter first if logically valid JDBC read needs parallelism Partition column + bounds + numPartitions Only some columns are needed from Parquet Column pruning Filter on partition columns Partition pruning`

MODULE 10 — ADVANCED SPARK: PLANS, UDFs & EXTERNAL SOURCES —

1. What does `df.explain()` help inspect?

- A. Execution plans
- B. Java constructors
- C. Only source files
- D. Database passwords

Answer: A — Execution plans

Explanation: explain shows logical/physical plan information.

2. What does Exchange commonly indicate?

- A. Data redistribution/shuffle
- B. Only filtering
- C. Regex compilation
- D. Object serialization only

Answer: A — Data redistribution/shuffle

Explanation: Exchange commonly represents repartitioning/data movement.

3. Why prefer built-in Spark SQL functions over an equivalent UDF?

- A. Spark can optimize/understand built-ins better
- B. UDFs are always faster
- C. Built-ins disable execution plans
- D. UDFs cannot run

Answer: A — Spark can optimize/understand built-ins better

Explanation: Built-ins expose semantics to Spark's optimizer.

4. Why filter before an expensive UDF when logically valid?

- A. Run UDF on fewer rows
- B. Increase shuffle
- C. Disable pruning
- D. Make all rows match

Answer: A — Run UDF on fewer rows

Explanation: Selective filtering reduces expensive custom computation.

5. What does `numPartitions` mean in a partitioned JDBC read?

- A. Number of JDBC read partitions/concurrent tasks
- B. Number of columns
- C. Number of database tables
- D. Number of UDFs

Answer: A — Number of JDBC read partitions/concurrent tasks

Explanation: It controls parallel read partitions/tasks and thus concurrency.

6. What is Parquet column pruning?

- A. Read only needed columns
- B. Read every column twice
- C. Drop all columns
- D. Sort columns alphabetically

Answer: A — Read only needed columns

Explanation: Columnar storage lets Spark avoid reading unused columns.

7. What can partition pruning reduce?

- A. Unnecessary partition reads
- B. Java inheritance
- C. Regex matches
- D. Thread count only

Answer: A — Unnecessary partition reads

Explanation: Filtering on partition columns can avoid reading irrelevant partitions.

8. Which raw-to-curved pattern is sensible?

- A. JSON ingest → transform/validate → Parquet-style analytics output
- B. Parquet → socket → regex only
- C. JVM → List → PDF only
- D. UDF → compile source

Answer: A — JSON ingest → transform/validate → Parquet-style analytics output

Explanation: JSON is convenient for flexible ingestion; Parquet is efficient for downstream analytics. MODULE 11 — STREAMING, FAULT TOLERANCE & DATA PROCESSING Exam target: Learn the rule, recognize the trap, then prove it by solving the questions without looking at the answers. HIGH-YIELD RULES ■ DStreams are the older RDD-based Spark Streaming abstraction; Structured Streaming uses DataFrame/Dataset structured APIs. ■ New DataFrame-style streaming applications generally favor Structured Streaming. ■ File streaming processes newly arriving files incrementally; files should appear atomically. ■ Checkpointing stores progress/state metadata needed for compatible recovery. ■ Checkpoint storage should be durable/reliable. ■ A window length is the amount of event history included; a slide interval controls how often the result advances/updates. ■ Watermarks help manage late event handling and bound state retention for event-time operations. ■ Socket sources are convenient demonstrations but are not durable production replay sources. CODE PATTERNS TO RECOGNIZE // Conceptual window 30-minute window 5-minute slide // includes 30 minutes of event history // advances/updates every 5 minutes COMMON MCQ TRAPS ■ Checkpointing is not the same as cache/persist. ■ Do not select an ephemeral socket source when a scenario requires durable replay/recovery. ■ Window length and slide interval answer different questions. ■ Deep state-store internals are outside the stated preparation depth. STREAMING COMPARISONS Concept Remember DStreams / Structured Streaming Legacy RDD-based / structured DataFrame-Dataset API Window length / slide History included / update frequency Checkpoint / cache Recovery/progress / reuse of computed data Durable source / socket Replay/recovery capable / demo-oriented ephemeral source

MODULE 11 — STREAMING, FAULT TOLERANCE & DATA PROCESSING

1. Which is generally preferred for new DataFrame-style Spark streaming?

- A. DStreams
- B. Structured Streaming
- C. Socket API only
- D. RDD textFile

Answer: B — Structured Streaming

Explanation: Structured Streaming uses DataFrame/Dataset structured APIs.

2. Why store streaming checkpoints durably?

- A. So progress/state can survive failure and resume
- B. To sort data
- C. To create a JVM
- D. To remove all state

Answer: A — So progress/state can survive failure and resume

Explanation: Checkpoint metadata/state supports recovery.

3. A 30-minute window updated every 5 minutes means:

- A. Window length 30m, slide 5m
- B. Window 5m, slide 30m
- C. Both 30m
- D. Both 5m

Answer: A — Window length 30m, slide 5m

Explanation: Window length is the history included; slide is update advancement.

4. What do watermarks help manage?

- A. Late event handling and bounded state retention
- B. Java overloading
- C. JDBC commits
- D. RDD compilation

Answer: A — Late event handling and bounded state retention

Explanation: Watermarks bound late-data/state handling for event-time operations.

5. Why is a socket source commonly considered a demo source?

- A. It is ephemeral and not a durable replay source
- B. It is always encrypted
- C. It is Parquet-based
- D. It guarantees recovery

Answer: A — It is ephemeral and not a durable replay source

Explanation: Socket sources are convenient demos but lack durable production replay characteristics.

6. New CSV/JSON files arrive throughout the day. Which streaming source fits?

- A. File streaming
- B. Java constructor
- C. HashSet
- D. HikariCP

Answer: A — File streaming

Explanation: Structured Streaming can process newly arriving files incrementally.

7. Checkpointing differs from cache/persist because checkpointing primarily supports:

- A. Recovery/progress metadata and lineage truncation in relevant contexts
- B. Only faster repeated actions
- C. Only sorting
- D. Only joins

Answer: A — Recovery/progress metadata and lineage truncation in relevant contexts

Explanation: Checkpointing is for durable recovery/progress rather than simply reuse.

8. What is the key conceptual difference between DStreams and Structured Streaming?

- A. DStreams are legacy RDD-based; Structured Streaming uses structured DataFrame/Dataset APIs
- B. DStreams are JDBC pools
- C. Structured Streaming is only Java
- D. They are identical

Answer: A — DStreams are legacy RDD-based; Structured Streaming uses structured DataFrame/Dataset APIs

Explanation: This is the core distinction expected at the stated assessment depth. FINAL MASTER COMPARISON SHEET Pair Key distinction JDK vs JRE vs JVM Development tools / runtime environment / execution engine Stack vs Heap Method-frame/local mental model / created objects and arrays List vs Set vs Map Ordered duplicates / uniqueness / key-value lookup Overloading vs Overriding Compile-time different parameters / runtime inherited behavior replacement Inheritance vs Composition is-a / has-a val vs var No rebinding / reassignment allowed Array vs immutable List Mutable indexed / immutable map vs flatMap One result per input / flatten nested zero-or-many results Transformation vs Action Lazy RDD definition / triggers execution Narrow vs Wide Limited partition dependency / redistribution/shuffle collect vs take All records to driver / limited sample cache/persist vs checkpoint Reuse computed data / recovery/progress and lineage truncation where applicable RDD vs DataFrame Low-level distributed collection / schema-aware structured API matches vs find Whole input / subsequence search Option vs Try vs Either Absence / exception capture / explicit success-error information DStream vs Structured Streaming Legacy RDD-based / structured DataFrame-Dataset API Window vs Slide History length / update frequency Race vs Deadlock Timing-dependent incorrect result / circular waiting commit vs rollback Make transaction changes permanent / undo uncommitted work